

Архиватор на базе алгоритма Хаффмана

Овечкин Андрей Максимович

Группа: 25.Б72-мм

Кафедра: Системного программирования

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Сжатие данных без потерь — одна из фундаментальных задач информатики. Алгоритм Хаффмана [?], предложенный Дэвидом Хаффманом в 1952 году, строит оптимальный префиксный код для заданного распределения символов и остаётся основой многих современных форматов сжатия (Deflate в gzip и PNG, BWT + Huffman в bzip2).

Реализация этого алгоритма позволяет на практике освоить работу с бинарными деревьями, побитовыми операциями и проектирование двоичных форматов. Результатом является консольный архиватор, написанный на языке Rust, который сжимает произвольные бинарные файлы и восстанавливает их без потерь.

2. Постановка задачи

Целью работы является разработка консольного архиватора на языке Rust, реализующего сжатие без потерь с помощью байтового кодирования Хаффмана с каноническими кодами и обеспечивающего точное восстановление исходных данных.

Для достижения цели требуется решить следующие задачи:

1. Выполнить обзор предметной области: алгоритмы сжатия без потерь, существующие архиваторы, язык Rust.
2. Спроектировать решение: формат архива, архитектуру модулей, интерфейс командной строки.
3. Реализовать сжатие и распаковку.
4. Провести тестирование: интеграционные тесты, краевые случаи, обработка повреждённых архивов.

3. Обзор

3.1. Алгоритм Хаффмана

Алгоритм Хаффмана решает задачу построения оптимального префиксного кода для заданного распределения символов. Оптимальность понимается в смысле минимальной средневзвешенной длины кодового слова при условии, что код является префиксным (ни один код не является префиксом другого), что позволяет однозначно декодировать последовательность без разделителей.

Алгоритм является жадным и работает следующим образом:

1. Каждый символ исходного алфавита становится листом дерева, весом которого является частота символа.
2. Два узла с наименьшими весами объединяются в новый внутренний узел, вес которого равен сумме весов потомков.
3. Шаг 2 повторяется, пока не останется один корневой узел.

Ветви дерева кодируются битами: левый потомок — 0, правый — 1. Кодом символа является последовательность битов на пути от корня до листа. Чем чаще встречается символ, тем короче его код. Классический алгоритм является двухпроходным: первый проход по файлу собирает статистику частот, второй — выполняет кодирование.

3.2. Адаптивный алгоритм Хаффмана

Существует также однопроходная модификация — адаптивный алгоритм Хаффмана, предложенный Галлагером и Кнуттом. В нём дерево строится и обновляется на лету по мере чтения данных, что устраняет необходимость в предварительном подсчёте частот и хранении таблицы кодов в заголовке. Однако адаптивный вариант сложнее в реализации из-за необходимости поддерживать инвариант дерева при каждом добавлении символа, а его степень сжатия незначительно ниже из-за использования локальных, а не глобальных частот.

3.3. Канонические коды

Прямая передача дерева Хаффмана в заголовке архива требует записи его топологии либо полной таблицы кодов, что занимает значительный объём. Эту проблему решают канонические коды: достаточно сохранить только длины кодов для каждого символа, а сами коды восстанавливаются по единому правилу. Пары «(длина, символ)» сортируются по возрастанию длины, затем символа. Первый код принимается равным нулю, а каждый следующий вычисляется по формуле:

$$\text{code}_i = (\text{code}_{i-1} + 1) \ll (\text{length}_i - \text{length}_{i-1})$$

3.4. Выводы из обзора

На основе анализа алгоритма были приняты следующие проектные решения:

- выбран классический двухпроходный алгоритм Хаффмана как наиболее наглядный для учебной реализации;
- применены канонические коды для минимизации заголовка архива;
- реализация выполнена на языке Rust;

4. Описание предлагаемого решения

4.1. Архитектура

Проект состоит из пяти модулей:

- `main.rs` — CLI: парсинг аргументов командной строки, запуск `zip` или `unzip`;
- `lib.rs` — публичное API: реэкспорт функций `zip` и `unzip`;
- `tree_lib.rs` — структура бинарного дерева `Node<T, U>` и обход DFS;
- `huffman_utils.rs` — построение дерева Хаффмана, генерация канонических кодов, восстановление дерева по таблице;
- `zip_lib.rs` — логика сжатия и распаковки.

4.2. Формат архива

Заголовок архива имеет следующую структуру:

Таблица 1: Формат заголовка архива		
Смещение	Размер	Описание
0	8 байт	Размер исходного файла (little-endian u64)
8	1 байт	Количество уникальных байтов минус один
9	переменный	Таблица длин кодов
...	...	Сжатые данные (побитовая упаковка)

Таблица длин кодов хранится в одном из двух форматов: если количество уникальных символов меньше 128, каждый элемент — пара (байт, длина) (компактный режим). Иначе записываются 256 байт, где позиция i содержит длину кода для символа i (значение 0 означает отсутствие символа). Такой подход минимизирует заголовочные данные при любом размере алфавита.

4.3. Алгоритм сжатия

Сжатие состоит из четырёх этапов.

1. Подсчёт частот. Входной файл читается побайтово, для каждого байта подсчитывается частота вхождений.

2. Построение дерева Хаффмана. Из узлов-листьев с частотами собирается вектор, сортируемый по частоте. На каждой итерации два узла с минимальной частотой сливаются в один внутренний узел, который вставляется обратно с помощью бинарного поиска. В случае одного уникального байта добавляется фиктивный узел с частотой 0.

3. Получение канонических кодов. DFS-обход собирает глубины листьев (длины кодов). Пары «(длина, символ)» сортируются по длине, затем по символу. Первый код равен 0; каждый следующий вычисляется по формуле:

$$\text{code}_i = (\text{code}_{i-1} + 1) \ll (\text{length}_i - \text{length}_{i-1})$$

Канонические коды позволяют хранить в заголовке только длины, а не сами коды, что существенно уменьшает размер заголовка.

4. Запись заголовка и данных. В файл записываются размер оригинала, количество символов, таблица длин. Затем исходный файл читается повторно, для каждого байта его код побитово упаковывается в выходной поток; неполный последний байт дополняется нулями.

4.4. Алгоритм распаковки

Распаковка читает заголовок, восстанавливает таблицу канонических кодов и строит дерево: для каждого кода создаётся путь в дереве (0 — левый потомок, 1 — правый), лист помечается символом. Затем сжатые данные читаются побитово: спуск по дереву до листа даёт очередной байт, после чего указатель возвращается в корень. Процесс продолжается, пока не будет восстановлено ровно `original_size` байт.

4.5. Технические детали

- **Два режима таблицы.** При < 128 символах хранятся только существующие пары (компактно), при ≥ 128 — полный массив на 256 значений. Переключение по одному биту исключает избыточность.
- **Edge-case: один уникальный байт.** Алгоритм Хаффмана требует как минимум двух узлов для построения дерева; при одном байте искусственно добавляется нулевой узел, что гарантирует корректную работу кодирования.

4.6. Используемые технологии

Язык программирования	Rust, edition 2024
Система сборки	Cargo
CI	GitHub Actions (cargo test, clippy, fmt)

Таблица 2: Интеграционные тесты

Тест	Проверяемый сценарий
random_true_test	Round-trip: zip → unzip, совпадение с оригиналом
test_large_data	Сжатие 1 МБ однородных данных
test_empty_file	Ошибка при пустом файле
test_file_one_byte	Один уникальный байт
test_file_same_bytes	512 одинаковых байт
test_file_all_bytes_n_times	Все 256 байт с разной частотой
test_file_all_bytes	Все 256 байт по 1 разу
test_unzip_short_file_defect	Слишком короткий архив
test_unzip_file_damaged_table	Испорченная таблица
test_unzip_wrong_original_size	Неверный original_size

5. Тестирование

Для проверки корректности работы написаны 10 интеграционных тестов.

Тесты покрывают корректное сжатие и распаковку, граничные случаи (пустой файл, один символ, однородные данные, полный алфавит), а также обработку повреждённых входных данных.

В непрерывной интеграции (GitHub Actions) при каждом пуше выполняются:

- `cargo test ---verbose` — запуск всех тестов;
- `cargo fmt ---check` — проверка форматирования;
- `cargo clippy --- -D warnings` — статический анализ кода.

Для демонстрации эффективности сжатия были произведены замеры на различных типах данных. Замеры выполнялись на Apple M1 (8 ГБ RAM, macOS 26.5.1). Время замерялось как wall-clock время вызова программы. Для каждого сценария выполнено по 50 запусков с последующим усреднением.

Таблица 3: Результаты замеров сжатия (усреднено за 50 запусков)

Тип данных	Исходный	Сжатый	Коэфф.	Скорость (сжатие / распаковка)
<i>Тестовые сценарии из интеграционных тестов</i>				
random_true_test	29 Б	42 Б	0,69	0,003 с / 0,003 с
test_file_one_byte	1 Б	12 Б	0,08	0,003 с / 0,003 с
test_file_same_bytes	512 Б	75 Б	6,83	0,004 с / 0,004 с
test_file_all_bytes	256 Б	521 Б	0,49	0,004 с / 0,004 с
test_all_bytes_n_times	32 896 Б	32 145 Б	1,02	0,087 с / 0,070 с
test_large_data	1 048 576 Б	131 083 Б	8,00	1,163 с / 1,719 с
<i>Дополнительные сценарии</i>				
Маленький текст	29 Б	58 Б	0,50	0,003 с / 0,003 с
Средний текст	21 500 Б	9 481 Б	2,27	0,039 с / 0,042 с
Большой текст	360 000 Б	157 533 Б	2,29	0,592 с / 0,656 с
Однородные данные (0xAD, 1 МБ)	1 048 576 Б	131 083 Б	8,00	1,164 с / 1,727 с
Все 256 байт	256 Б	521 Б	0,49	0,004 с / 0,004 с
256 байт с частотой <i>i</i>	32 896 Б	32 145 Б	1,02	0,087 с / 0,070 с
Сильно смещённое распределение	999 980 Б	174 998 Б	5,71	1,189 с / 1,661 с
Случайные байты (100 КБ)	102 400 Б	102 665 Б	1,00	0,273 с / 0,216 с

Результаты отражают теоретические свойства алгоритма Хаффмана: наибольшая степень сжатия достигается на данных с сильным перекосом частот (однородные — 8,00; смещённое распределение — 5,71). Тексты сжимаются примерно вдвое (2,27–2,29). При равномерном распределении символов (случайные байты, полный алфавит) сжатие отсутствует, а маленькие файлы (30 Б) увеличиваются, что ожидаемо и не является дефектом реализации. Время сжатия и распаковки составляет доли секунды для файлов размером до сотен килобайт и 1–2 секунды для файлов размером 1 МБ.

6. Заключение

В ходе выполнения работы получены следующие результаты:

- Разработан консольный архиватор на языке Rust, реализующий сжатие без потерь с помощью байтового кодирования Хаффмана.
- Применены канонические коды, позволяющие хранить в заголовке только длины кодов и восстанавливать их на стороне декодера.
- Спроектирован бинарный формат архива с компактным и полным представлением таблицы длин.

- Проведено тестирование (10 интеграционных тестов, включая граничные случаи и повреждённые заголовки), настроена непрерывная интеграция с линтером и запуском тестов.
- Выполнены замеры степени сжатия на разных типах данных.

Репозиторий: <https://github.com/Iseyne/Archiver-Huffman>

Список литературы

[1] Huffman, D. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*. 40 (9): 1098–1101.